

Datum: 8. Juni 2026 | Projekt: VaadinExcelExport (github.com/MaKnoNet/VaadinExcelExport)

Worum geht es?
GridExcelExporter (de.makno.vaadinexcelexport.export) ist eine schlanke Bruecke: sie liest die Spalten des Vaadin-Grid und delegiert das Schreiben an die Bibliothek **xlsxbuilder** (de.makno.xlsxbuilder – XlsxBuilder + WorkbookBuilder, auf Apache POI SXSSF, out-of-core streamend). Flowingcodes **GridExporter** (com.floatingcode.vaadin.addons.gridexporter) baut direkt auf Apache POI auf, bietet mehrere Ausgabeformate und benoetigt eine Vaadin-Session.

1 Benchmark-Ergebnis (aus SQL-Datenbank)

Gemessen gegen eine file-basierte H2-Datenbank mit 25.000 Zeilen: GridExcelExporter streamt **out-of-core** aus einem forward-only JDBC-ResultSet (FetchSize 1000); Flowingcode liest ueber das lazy, seitenweise Grid (Apache POI im RAM). 2 Warmup- und 5 Messlaeufe.

Engine	Median	Avg	Zeilen/s	Groesse
GridExcelExporter <i>auf xlsxbuilder</i>	358 ms	361 ms	69.747	2.143 KB
GridExporter <i>Flowingcode</i>	10.182 ms	9.805 ms	2.455	2.146 KB

GridExcelExporter ist ~28x schneller

Konfiguration: Java 21, Gradle 8.9, Vaadin 24.5.3, Apache POI 5.4.0, H2 2.3.232 | Ausfuehrung: ./gradlew benchmark [-Prows=N] | Klasse: ExcelExporterBenchmarkTest (misst beide Engines aus der H2-Datenbank). Der optionale Pipeline-Parallelismus (parallel()) bringt bei POI-dominierter Last wenig – sein Nutzen liegt bei einem Producer-Flaschenhals.

2 Feature-Vergleich

2.1 Ausgabeformate

Feature	GridExcelExporter auf xlsxbuilder	GridExporter Flowingcode
Excel (.xlsx)	✓	✓
CSV	✗	✓
DOCX (Word)	✗	✓
PDF	✗	✓

xlsxbuilder erzeugt bewusst nur .xlsx: CSV wurde wieder entfernt, weil sich FORMULA-/HYPERLINK-Zellen in CSV nicht abbilden lassen, ohne ein vollstaendiges In-Memory-Workbook auszuwerten (Konflikt mit dem Out-of-core-Design). DOCX/PDF bietet nur Flowingcode.

2.2 Datenquelle, Sortierung & Filter

Feature	GridExcelExporter auf xlsxbuilder	GridExporter Flowingcode
Respektiert Grid-Sortierung (Spaltenkoepfe)	✓	✓
Respektiert aktiven Grid-Filter automatisch	✓	✓

Feature	GridExcelExporter auf xlsxbuilder	GridExporter Flowingcode
Programmatischer Daten-Filter (Predicate)	✓	✗
Out-of-core-Stream direkt aus JDBC-ResultSet	✓	✗
Out-of-core External-Merge-Sort (sortBy)	✓	✗
Streaming / out-of-core (konstanter Speicher)	✓	✗
TreeGrid / hierarchische Daten	✗	✓

Beide Exporte spiegeln den aktiven Grid-Filter automatisch: Flowingcode liest den Filter aus dem DataProvider; GridExcelExporter erhält den Suchbegriff als parametrisiertes SQL-WHERE und streamt nur die passenden Zeilen weiterhin out-of-core (ein Suchfeld, eine Quelle der Wahrheit fuer Anzeige + beide Exporte).

2.3 Zelltypen & Formatierung

Feature	GridExcelExporter auf xlsxbuilder	GridExporter Flowingcode
Typisierte Zellen (INTEGER, LONG, DOUBLE, DECIMAL, BOOLEAN, DATE, DATETIME, TIME)	✓	✗
Echte Excel-Formeln (z. B. =E2*0.19)	✓	✗
Klickbare Hyperlinks in Excel	HYPERLINK	URL-Text
Excel-Formatcode pro Spalte (#,##0.00)	✓	✓
Java DecimalFormat/DateFormat + Excel-Code kombiniert	✗	✓
Null-Wert-Handler (Platzhalter-Text / leere Zelle)	✓	✓

2.4 Layout, Footer & Aggregation

Feature	GridExcelExporter auf xlsxbuilder	GridExporter Flowingcode
Einfacher Sheet-Name	✓	✓
Titelzeile ueber alle Spalten, auto-merge (header)	✓	✓
Fusszeile ueber alle Spalten, auto-merge (footer)	✓	✓
Berechnete Footer-Platzhalter {rowCount} / {sum:Spalte} / {datetime}	✓	✗
Summenzeile (sumColumn, optional als Excel-Formel)	✓	~
Auto-Size Columns (beim Streamen gemessen)	✓	✓
Mehrzeilige Header / Joined Headers	✓	✓
Spalten-Reihenfolge im Export ueberschreiben	✓	✓
Template-basierter Export (.xlsx-Vorlage)	✗	✓
Eigene Platzhalter im Template	✗	✓

Neu in xlsxbuilder: Titel-/Fusszeilen (ueber die Breite gemerged) mit Platzhaltern, die beim Schreiben aufgeloeset werden – eingebaut {date}/{datetime}, sowie im Footer {rowCount} und {sum:Spaltenname}; die Summenzeile aktiviert die Summenverfolgung. Die Demo nutzt eine Fusszeile „Erzeugt am {datetime} – {rowCount} Zeilen – Summe Betrag: {sum:Betrag} EUR“. Verbundene Header (joined headers) entstehen aus gruppierten Spalten (columnGroups(...), gemergte Gruppenzeile ueber dem Spaltenkopf); die Export-Spaltenreihenfolge laesst sich unabhaengig von der Anzeige ueberschreiben (GridExcelExporter.from(sheet, grid,

columnOrder)).

2.5 UI-Integration, Server-Tuning & Performance

Feature	GridExcelExporter auf xlsxbuilder	GridExporter Flowingcode
Fertige Download-Buttons (auto an Grid-Footer)	✗	✓
Concurrent-Download-Throttling (Semaphore)	✗	✓
Pipeline-Parallelismus (parallel())	✓	✗
Server-Tuning: Temp-Verzeichnis + SXSSF-Fenster	✓	✗
Single-Use-Guard (Schutz vor Wiederverwendung)	✓	✗
Performance (25.000 Zeilen aus DB, Median)	358 ms	10.182 ms
Arbeitsspeicher	Streaming	RAM
Benoetigt Vaadin-Session/UI	Nein	Ja

3 Empfehlung

GridExcelExporter (xlsxbuilder)

- Performance zaehlt (~28x schneller)
- Sehr grosse Datenmengen out-of-core streamen (direkt aus der DB)
- Echte typisierte Zellen, Formeln, klickbare Hyperlinks
- Sortierung der Tabelle wird uebernommen
- Fusszeile/Summenzeile mit Platzhaltern ({rowCount}, {sum:Spalte})

GridExporter (Flowingcode)

- Hierarchische Daten (TreeGrid) exportieren
- Mehrere Formate anbieten (CSV, DOCX, PDF)
- Excel-Vorlage mit Platzhaltern befuellen
- Mehrzeilige / verbundene Header
- Fertige UI-Buttons ohne eigenen Code

4 Technische Details

Projekt	VaadinExcelExport (github.com/MaKnoNet/VaadinExcelExport)
Stack	Java 21, Spring Boot 3.3.5, Vaadin 24.5.3
Brueckenklasse	de.makno.vaadinexcelexport.export.GridExcelExporter (liest Grid-Spalten, delegiert an xlsxbuilder)
xlsxbuilder	de.makno.xlsxbuilder:xlsxbuilder:1.0.0 - XlsxBuilder + WorkbookBuilder (Apache POI SXSSF), Gradle Composite Build
Flowingcode	com.floatingcode.vaadin.addons.gridexporter.GridExporter (org.vaadin.addons.floatingcode.grid-exporter-addon:2.5.0)
Testdaten	H2 2.3.232 (file-basiert, reines JDBC) - lazy geladen / gestreamt
Apache POI	5.4.0
Benchmark	ExcelExporterBenchmarkTest (@Tag("benchmark"), aus H2)
Ausfuehrung	./gradlew benchmark [-Prows=N]
Datum	8. Juni 2026